

第九章 虚拟内存：动态内存分配

现代内存分配器

教 师：邹翔宇

计算机科学与技术学院

哈尔滨工业大学（深圳）

malloc: memory allocator

主要内容

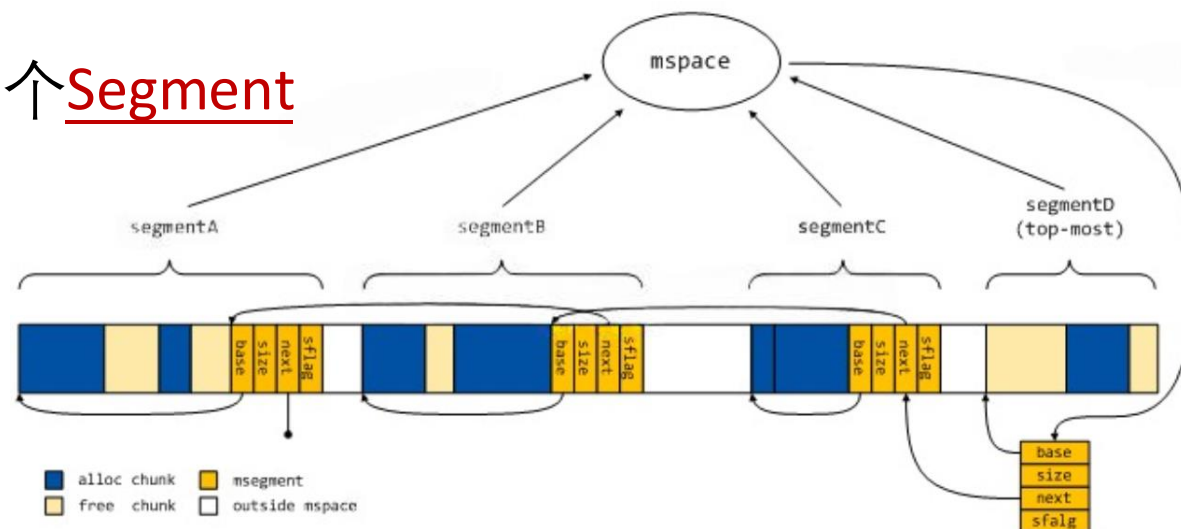
- dlmalloc——Doug Lea's malloc
- ptmalloc——Pthreads malloc
- tcmalloc——Thread-Caching malloc
- jemalloc——Jason Evans' malloc

dlmalloc

- Doug Lea's malloc——1996
- 现代malloc的基础实现
- 核心数据结构
 - Chunk（块）、Segment（段）
 - Segment包含多个Chunk
- Segment：与操作系统交互的单位
- Chunk：与用户交互的单位
- 核心思想：申请一大片内存，按需发给用户

dlmalloc: 如何从OS获取内存空间

- dlmalloc获取的内存空间并不完全来自“堆” (heap)
 - 甚至来自堆的比例越来越少
- 原因: OS大量使用mmap命令占用位置, 堆的延伸可能受阻
 - 动态库、内存映射、线程栈等
- 应对方法: “打不过就加入”, 也使用mmap获取空间
- 后果: 每次获取的空间可能都不连续
- 对应数据结构: 每个连续的空间对应一个Segment



dlmalloc: 如何响应用户需求

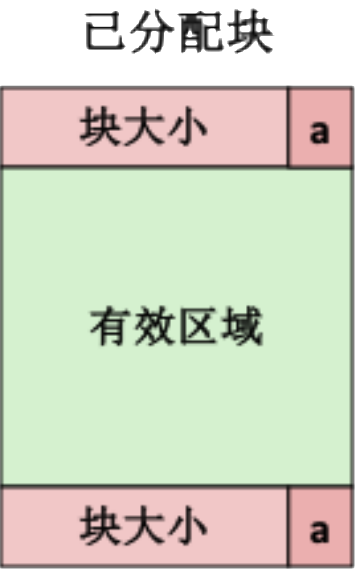
- 响应用户需求以Chunk为单位
- 每个Segment一开始就是一个超大的Chunk
- 找出能满足用户需求的Chunk
 - 如果Chunk过大, 则拆分
- 两个核心机制:
 - 如何在Segment中标记出每个Chunk的边界?
 - Chunk的组织结构
 - 如何快速找出能满足用户需求的Chunk?
 - Chunk的索引

dldmalloc: Chunk的组织结构

- 显式链表结构

```
2112 A chunk that's in use looks like:
2113
2114 chunk-> +-----+
2115         | Size of previous chunk (if P = 0) |
2116         +-----+
2117         +-----+ |P|
2118         | Size of this chunk | 1| +-+
2119 mem-> +-----+
2120         | |
2121         +-+ +-+
2122         | |
2123         +-+ +-+
2124         | |
2125         +-+ size - sizeof(size_t) available payload bytes +-+
2126         : |
2127 chunk-> +-+ +-+
2128         | |
2129         +-----+ |1|
2130         +-----+
2131         | Size of next chunk (may or may not be in use) | +-+
2132 mem-> +-----+
```

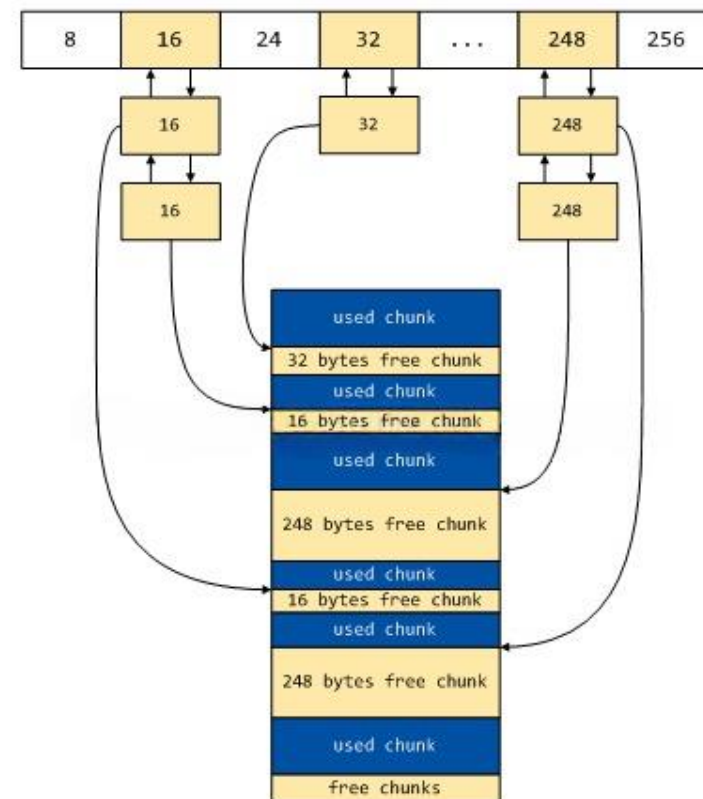
```
2136 chunk-> +-+ +-+
2137         | User payload (must be in use, or we would have merged!) |
2138         +-----+
2139         +-----+ |P|
2140         | Size of this chunk | 0| +-+
2141 mem-> +-----+
2142         | Next pointer |
2143         +-----+
2144         | Prev pointer |
2145         +-----+
2146         | :
2147         +-+ size - sizeof(struct chunk) unused bytes +-+
2148         : |
2149 chunk-> +-----+
2150         | Size of this chunk |
2151         +-----+
2152         +-----+ |0|
2153         | Size of next chunk (must be in use, or we would have merged)| +-+
2154 mem-> +-----+
2155         | :
2156         +-+ User payload +-+
2157         : |
2158         +-----+
2159         |0|
2160         +-+
```



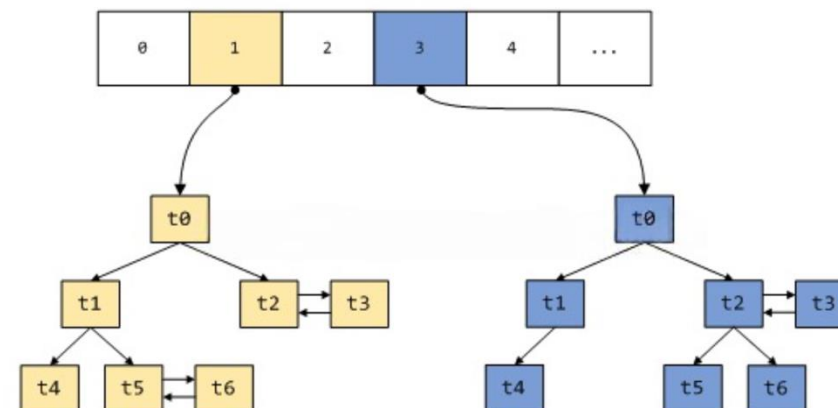
“misleading but accurate and necessary” —— 作者

dlmalloc: 空闲Chunk的索引

- 介于first-fit和best-fit之间
- 基于bin（分桶）的Chunk索引
 - 将Chunk按照大小组织起来
- Small bin (≤ 256 字节)
 - 以8字节为间隔，Chunk大小组织hash表
- Tree bin (≥ 256 字节)
 - 二级索引，先分桶，再用DST树组织



箱号	区间范围	区间长度
0	[256, 384)	128
1	[384, 512)	128
2	[512, 768)	256
3	[768, 1024)	256
4	[1024, 1536)	512
5	[1536, 2048)	512



dlmalloc: 如何回收空间

- 用户free: 直接更新Chunk组织中的标记位、空闲Chunk索引
- 向OS交还内存空间
 - 调整栈顶指针
 - munmap、mremap
 - 只能交还连续的、未使用的内存

dlmalloc: 缺点

- 面向单线程编程
 - 不支持多线程并发
 - 多线程调用malloc需要加全局锁, 性能较差
- 空闲Chunk索引机制较为复杂, 多线程性能较差
- 多用于嵌入式系统

malloc: memory allocator

主要内容

- dlmalloc——Doug Lea's malloc
- ptmalloc——Pthreads malloc
- tcmalloc——Thread-Caching malloc
- jemalloc——Jason Evans' malloc

ptmalloc

- ptmalloc2: glibc 2.26+中的默认实现
- 在dlmalloc的基础上，面向多线程优化
- 主要的新机制：arena，tcache
- arena：全功能的内存管理子单元
- tcache：线程独占的可用内存空间
- 核心思想：尽量对每个线程进行独立的内存分配，避免线程间交互

ptmalloc: arena

- 可以简单理解：每个arena都是一个独立的dlmalloc
 - 访问初始化时，堆内存对应main arena，由主线程持有
 - 当某个线程访问arena被锁阻塞，则
 - 尝试建立新的arena (thread arena)
 - 每个arena都通过mmap建立
- 理想情况下，每个线程对应一个arena
 - 为了避免内存浪费过多，arena的数量上限为CPU核数*8
 - 当访问arena受阻，且arena数量达成上限，则尝试访问其他arena
- 最终效果：降低了锁的粒度 (arena级)

ptmalloc: tcache

- 每个线程私有的小内存池
- 解决的问题：小对象频繁申请释放（web server、数据库中每秒数百万次 malloc）
- 具体组织方式类似于small bin
- 设置不同大小的Chunk，如16B、32B、64B、128B、256B等
 - 每种大小的Chunk准备7-20个
 - 只供当前线程使用
 - 无锁
- 最终效果：避免arena的竞争

malloc: memory allocator

主要内容

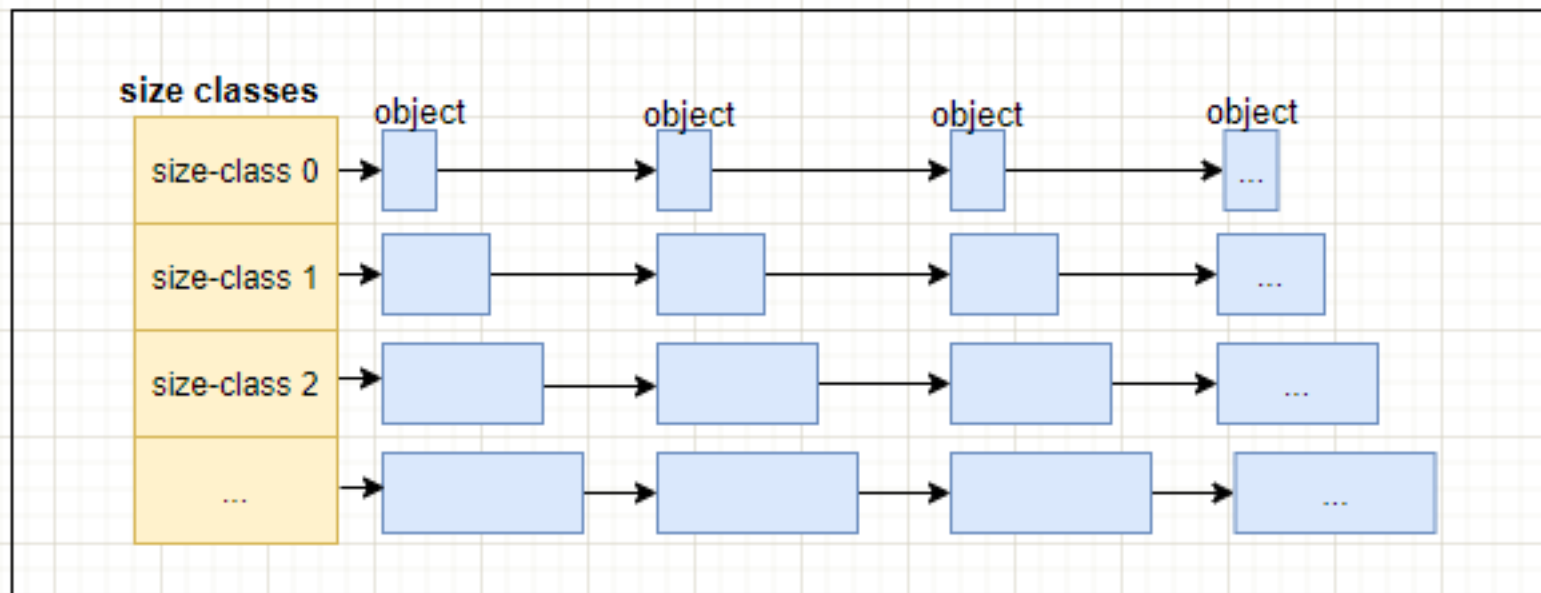
- dlmalloc——Doug Lea's malloc
- ptmalloc——Pthreads malloc
- tcmalloc——Thread-Caching malloc
- jemalloc——Jason Evans' malloc

tcmalloc

- tcmalloc: google的malloc实现
- 面向大规模并行计算
- 相对不计较内存浪费, Chunk的大小固定为8B、16B、32B、64B...
- 核心机制: ThreadCache、PageHeap
- ThreadCache: 线程独占的内存块集合 (已切为固定大小)
- PageHeap: 未切块的内存区域
- 核心思想: 不考虑内存碎片问题, 性能为先

tcmalloc: ThreadCache

- 与ptmalloc的tcache机制相同
 - tcache是ptmalloc2中新加入的机制，ptmalloc1中只有arena机制
 - tcmalloc比ptmalloc2更早
- 线程私有的内存块集合，无锁



tcmalloc: PageHeap

- 分配器向OS预申请的内存页
 - 通过mmap, 几乎不操作进程的堆 (heap, sbrk)
- 用户申请大对象时, 直接从PageHeap提供对应的整数个页面
- 某个线程ThreadCache容量不足时,
 - 从PageHeap获取大量页面, 切分为不同大小的块提供给ThreadCache
- 访问PageHeap时需要加锁, 但是每次返回批量的块, 调用次数少

tcmalloc: 优缺点

- 适合小对象的高频申请释放（无锁）
- 大对象申请释放效率不好（全局锁）
- 内存利用率较低
 - 没有外部碎片（所有块的大小都是对齐的）
 - 内部碎片较多（返回的空间往往比申请的要大不少）
- Google后续还开发了scudo分配器，主要聚焦于内存安全

malloc: memory allocator

主要内容

- dlmalloc——Doug Lea's malloc
- ptmalloc——Pthreads malloc
- tcmalloc——Thread-Caching malloc
- jemalloc——Jason Evans' malloc

jemalloc

- jemalloc: Jason Evans——2005
- 应用于FreeBSD, Firefox, Redis, Android等
- 集各家所长, 机制较为复杂
- 核心机制: 网格化的内存管理、arena、tcache、内存监控模块
- 网格化的内存管理: 只分配固定大小的内存空间
- arena、tcache: 与ptmalloc类似
- 内存监控模块:

jemalloc: 组织结构

Arena

└─ Chunk

└─ Run

└─ Region

└─ Region

└─ Region

└─ Run

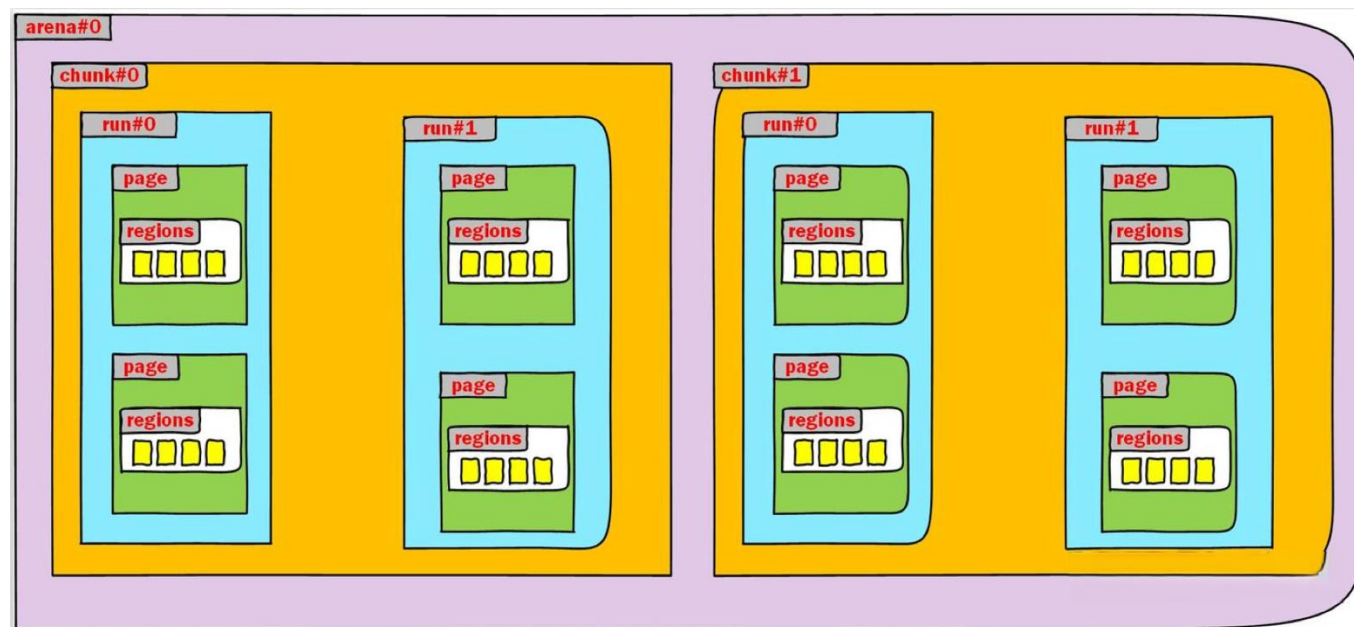
└─ Run

线程独占，或少量线程共享

补充容量的基本单位

不同大小小对象的组织单位

返回给用户的小对象，比如16B、32B



jemalloc: 组织结构

- Run中包含若干相同大小的region
- region大小相同，管理简便
- 通过bitmap来管理空闲/占用的region
 - 占用与释放是需要修改bitmap，不涉及合并、拆分
- 相同大小的小对象连续存放，对CPU cache访问友好
- jemalloc将内存归还给OS时，以run为基本单位
 - 只有整个run都是空闲时，才可归还

jemalloc: 丰富的实时监控和调控功能

- `int mallctl(const char *name, void *oldp, size_t *oldlenp, void *newp, size_t newlen)`: jemalloc控制接口
- `name`: 要查询或设置的路径 (类似 `/arena/0/allocated`)
- `oldp/oldlenp`: 用于读出当前值
- `newp/newlen`: 用于设置新值

<code>/stats/allocated</code>	当前程序实际分配出去的总内存字节数
<code>/stats/resident</code>	当前程序占用的物理内存大小 (RSS)
<code>/stats/active</code>	jemalloc 认为仍在活跃使用的内存
<code>/stats/mapped</code>	jemalloc 通过 <code>mmap</code> 总共映射的虚拟内存量
<code>/stats/metadata</code>	元数据开销大小
<code>/stats/arenas.N.*</code>	每个 Arena 的详细信息, 包括 <code>allocated</code> , <code>dirty pages</code> 等

jemalloc: 丰富的实时监控和调控功能

- 动态可修改的参数

/opt/dirty_decay_ms	空闲 dirty页延迟归还时间（控制内存回收策略）
/opt/muzzy_decay_ms	空闲 muzzy页延迟归还时间
/opt/metadata_thp	是否启用 Transparent Huge Page（THP）支持
/opt/arenas	最大 Arena 数量（线程过多时调大提高并发）
/opt/tcache	是否启用线程局部缓存（tcache）
/opt/lg_tcache_max	tcache支持的最大对象大小对数（默认32KB）

*Hope you
enjoyed
the
CSAPP
course!*